BLOCKSEC

Security Audit Report for DStock Peripheral

Date: January 21, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**

1.1 About the Audit Target . . . 1

1.2 Disclaimer . . . 1

1.3 Procedure of Auditing . . . 2

1.3.1 Security Issues . . . 2

1.3.2 Additional Recommendation . . . 3

1.4 Security Model . . . 3

Chapter 2 Findings **4**

2.1 Security Issue . . . 4

2.1.1 Potential DoS in functions `wrapAndBridge()` and `wrapAndBridgeNative()` . . . 4

2.1.2 Incorrect native refund logic in function `refundNative()` . . . 5

2.1.3 Lack of slippage protection . . . 6

2.1.4 Lack of revoking approvals . . . 7

2.1.5 Incorrect partial unwrap logic . . . 8

2.1.6 Excessive refund due to rounding . . . 10

2.2 Recommendation . . . 10

2.2.1 Lack of invoking function `_disableInitializers()` . . . 10

2.2.2 Add access control to `unwrapAndPayout()` . . . 10

2.2.3 Add a quoter for function `wrapAndBridgeNative()` . . . 11

2.2.4 Adopt accurate variable naming . . . 12

2.2.5 Revise the misleading annotation . . . 12

2.2.6 Correct length of storage gaps . . . 13

2.3 Note . . . 14

2.3.1 Potential centralization risks . . . 14

2.3.2 Correct implementation and configuration of adapters . . . 14

2.3.3 Risks regarding rebasing assets . . . 14

Report Manifest

Item	Description
Client	DStock
Target	DStock Peripheral

Version History

Version	Date	Description
1.0	January 21, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-01-21 07:12:35+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of DStock Peripheral of DStock.

The DStock Peripheral contracts provide LayerZero-integrated bridge and router functionality for cross-chain token operations. This project allows users to wrap native gas tokens (BNB/ETH) or underlying tokens (native ERC20 or OFT assets) into a DStock wrapper and bridge them to the target chain. Meanwhile, the project provides a compose callback feature to process bridged tokens further, which is divided into two types: forward compose (wrapping the underlying token into share tokens and bridging to the target chain) and reverse compose (receiving share tokens, unwrapping them into the underlying tokens, and delivering them locally or continuing to bridge).

Note this audit only focuses on the smart contracts in the following directories/files:

- src

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
dstock-peripheral	`Version 1`	`c97223a0cfbe31ea53fd7bf06e4240bfa0c495fd`
	`Version 2`	`d46bab4a47033197ad96b952c1fb54fa252c5639`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset.

[1] https://github.com/dstockofficial/dstock-peripheral

Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism

* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **six** potential security issues. Besides, we have **six** recommendations and **three** notes.

- High Risk: 1
- Medium Risk: 4
- Low Risk: 1
- Recommendation: 6
- Note: 3

ID	Severity	Description	Category	Status
1	High	Potential DoS in functions `wrapAndBridge()` and `wrapAndBridgeNative()`	Security Issue	Fixed
2	Medium	Incorrect native refund logic in function `refundNative()`	Security Issue	Fixed
3	Medium	Lack of slippage protection	Security Issue	Fixed
4	Medium	Lack of revoking approvals	Security Issue	Fixed
5	Medium	Incorrect partial unwrap logic	Security Issue	Fixed
6	Low	Excessive refund due to rounding	Security Issue	Fixed
7	-	Lack of invoking function `_disableInitializers()`	Recommendation	Fixed
8	-	Add access control to `unwrapAndPayout()`	Recommendation	Fixed
9	-	Add a quoter for function `wrapAndBridgeNative()`	Recommendation	Fixed
10	-	Adopt accurate variable naming	Recommendation	Fixed
11	-	Revise the misleading annotation	Recommendation	Fixed
12	-	Correct length of storage gaps	Recommendation	Fixed
13	-	Potential centralization risks	Note	-
14	-	Correct implementation and configuration of adapters	Note	-
15	-	Risks regarding rebasing assets	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential DoS in functions `wrapAndBridge()` and `wrapAndBridgeNative()`

Severity High

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter` [1], the functions `wrapAndBridge()` and `wrapAndBridgeNative()` invoke function `shareAdapter.send()` and set the parameter `minAmountLD` to the same as `amountSentLD`. However, the current design is incompatible with the standard LayerZero OFT implementation. Specifically, the LayerZero OFT adapter performs an internal dust-removal operation that slightly reduces the effective transferred amount via function `_removeDust()` [2], and requires the final amount be greater than `minAmountLD`. Bridging tokens with amounts beyond the shared decimals cannot satisfy this requirement and causes the transaction to revert.

```
    IOFTLike.SendParam memory sp = IOFTLike.SendParam({
        dstEid: dstEid,
        to: to,
        amountLD: amountSentLD,
        minAmountLD: amountSentLD,
        extraOptions: extraOptions,
        composeMsg: "",
        oftCmd: ""
    });
```

Listing 2.1: src/DStockComposerRouter.sol

Impact Any wrap operation containing fractional digits results in cross-chain bridging failure.

Suggestion Revise the code logic accordingly.

2.1.2 Incorrect native refund logic in function `refundNative()`

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, function `refundNative()` is designed to refund leftover native fees from the current LayerZero compose execution to the caller. However, the function transfers the entire contract balance to the `refundBsc` address, which could be larger than per-message fee refunds. If the contract holds native assets, they are sent to users, leading to excessive refunds and potential fund loss.

```
function _refundNative(address to) internal {
    if (to == address(0)) return;
    uint256 bal = address(this).balance;
    if (bal == 0) return;
    (bool ok, ) = to.call{value: bal}("");
    ok;
}
```

Listing 2.2: src/DStockComposerRouter.sol

[1] Note that the contract name is changed to `DStockComposerRouterV2` in `Version 2`.

[2] https://github.com/LayerZero-Labs/LayerZero-v2/blob/ab9b083410b9359285a5756807e1b6145d4711a7/packages/layerzero-v2/evm/oapp/contracts/oft/OFTCore.sol#L317

```
function _lzComposeReverse(address wrapper, bytes32 guid, uint256 sharesIn, bytes memory inner
    ) internal {
    ReverseRouteMsg memory r;
    try this._decodeReverseRouteMsg(inner) returns (ReverseRouteMsg memory rr) {
        r = rr;
    } catch {
        _fail(guid, "decode_reverse_route_failed", address(0), sharesIn);
        return;
    }
    if (r.refundBsc == address(0)) {
        _fail(guid, "refund_zero", address(0), sharesIn);
        return;
    }
```

Listing 2.3: src/DStockComposerRouter.sol

Impact Users could receive excessive refunds, potentially leading to a loss of the router.

Suggestion Refund only the native amount attributable to the current execution context.

2.1.3 Lack of slippage protection

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, functions `wrapAndBridge()` and `wrapAndBridgeNative()` allow users to wrap underlying tokens via the function `wrap()`. However, this function does not allow users to specify any slippage protection parameter. The wrap operations are executed at the spot price at the moment of transaction confirmation. As a result, this design may lead users to receive fewer shares than expected, potentially resulting in financial losses when the wrapper exchange rate changes unfavorably.

```
function wrapAndBridgeNative(
    uint256 amountNative,
    uint32 dstEid,
    bytes32 to,
    bytes calldata extraOptions
) external payable nonReentrant returns (uint256 amountSentLD) {
    if (amountNative == 0) revert AmountZero();
    if (to == bytes32(0)) revert InvalidRecipient();

    address w = wrappedNative;
    if (w == address(0)) revert WrappedNativeNotSet();
    if (msg.value < amountNative) revert InsufficientFee(msg.value, amountNative);

    // 1) Wrap native into ERC20 on this router.
    IWrappedNative(w).deposit{value: amountNative}();

    // 2) Reuse the same wrap + bridge pipeline, using `w` as the underlying token.
    address wrapper = underlyingToWrapper[w];
    address shareAdapter = underlyingToShareAdapter[w];
```

```
    if (wrapper == address(0) || shareAdapter == address(0)) revert InvalidOApp();

    IERC20(w).forceApprove(wrapper, amountNative);
    (uint256 net18, ) = IDStockWrapperLike(wrapper).wrap(w, amountNative, address(this));
    amountSentLD = net18;
```

Listing 2.4: src/DStockComposerRouter.sol

```
function wrapAndBridge(
    address underlying,
    uint256 amount,
    uint32 dstEid,
    bytes32 to,
    bytes calldata extraOptions
) external payable nonReentrant returns (uint256 amountSentLD) {
    if (amount == 0) revert AmountZero();
    if (to == bytes32(0)) revert InvalidRecipient();

    address wrapper = underlyingToWrapper[underlying];
    address shareAdapter = underlyingToShareAdapter[underlying];
    if (wrapper == address(0) || shareAdapter == address(0)) revert InvalidOApp();

    IERC20(underlying).safeTransferFrom(msg.sender, address(this), amount);
    IERC20(underlying).forceApprove(wrapper, amount);

    (uint256 net18, ) = IDStockWrapperLike(wrapper).wrap(underlying, amount, address(this));
    amountSentLD = net18;
```

Listing 2.5: src/DStockComposerRouter.sol

Impact Users may receive fewer shares than expected when wrapping tokens.

Suggestion It is recommended to add a slippage parameter to these functions and the corresponding validation.

2.1.4 Lack of revoking approvals

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In functions `wrapAndBridge()`, `wrapAndBridgeNative()`, and `_sendShares()`, tokens are approved to the `shareAdapter` to perform cross-chain transfers. However, these functions fail to revoke the approval after interaction with the adapter. If the adapter provides a path to spend approved tokens on behalf of others, this residual approval can be abused to drain funds from the contract `DStockComposerRouter`.

```
    IERC20(wrapper).forceApprove(shareAdapter, amountSentLD);

    IOFTLike.SendParam memory sp = IOFTLike.SendParam({
        dstEid: dstEid,
        to: to,
```

```
        amountLD: amountSentLD,
        minAmountLD: amountSentLD,
        extraOptions: extraOptions,
        composeMsg: "",
        oftCmd: ""
    });

    IOFTLike.MessagingFee memory fee = IOFTLike(shareAdapter).quoteSend(sp, false);
    if (msg.value < fee.nativeFee) revert InsufficientFee(msg.value, fee.nativeFee);
```

Listing 2.6: src/DStockComposerRouter.sol

```
    IERC20(wrapper).forceApprove(shareAdapter, amountSentLD);

    IOFTLike.SendParam memory sp = IOFTLike.SendParam({
        dstEid: dstEid,
        to: to,
        amountLD: amountSentLD,
        minAmountLD: amountSentLD,
        extraOptions: extraOptions,
        composeMsg: "",
        oftCmd: ""
    });

    IOFTLike.MessagingFee memory fee = IOFTLike(shareAdapter).quoteSend(sp, false);
    uint256 availableFee = msg.value - amountNative;
    if (availableFee < fee.nativeFee) revert InsufficientFee(availableFee, fee.nativeFee);

    IOFTLike(shareAdapter).send{value: fee.nativeFee}(sp, fee, msg.sender);
```

Listing 2.7: src/DStockComposerRouter.sol

```
    IERC20(wrapper).forceApprove(shareAdapter, sharesOut);

    try IOFTLike(shareAdapter).send{value: fee2.nativeFee}(sp, fee2, refundBsc) {
        emit WrappedAndForwarded(guid, finalDstEid, finalTo, underlyingIn, sharesOut);
        return true;
```

Listing 2.8: src/DStockComposerRouter.sol

Impact Malicious actors may abuse residual allowances to drain funds from contract `DStockComposerRouter`.

Suggestion Revoke the allowance by invoking function `approve(shareAdapter, 0)` after the operation completes.

2.1.5 Incorrect partial unwrap logic

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, the function `_unwrapShares()` calculates the actual unwrapped amounts based on the parameter `unwrapBps`. When the specified `unwrapBps` is less than `10_000`, only a portion of the wrapper tokens are unwrapped into underlying tokens. However, the function fails to implement refund or collection logic for residual wrapper tokens.

```
function _unwrapShares(address wrapper, address underlying, bytes32 guid, uint256 sharesIn,
    address refundBsc, uint16 unwrapBps)
    internal
    returns (uint256)
{
    // Compose assumption: ‘sharesIn‘ shares should already be on this router.
    uint256 balShares = IERC20(wrapper).balanceOf(address(this));
    if (balShares < sharesIn) {
        _refundToken(wrapper, guid, "insufficient_shares", refundBsc, sharesIn);
        return 0;
    }

    // We may choose to unwrap only a fraction of shares (basis points).
    uint256 attemptUnderlying = (sharesIn * uint256(unwrapBps)) / 10_000;
    if (attemptUnderlying == 0) {
        _refundToken(wrapper, guid, "unwrap_zero_amount", refundBsc, sharesIn);
        return 0;
    }

    // Track underlying delta to compute actual output.
    uint256 underlyingBalBefore = IERC20(underlying).balanceOf(address(this));

    try IDStockWrapperLike(wrapper).unwrap(underlying, attemptUnderlying, address(this)) {}
        catch {
        _refundToken(wrapper, guid, "unwrap_failed", refundBsc, sharesIn);
        return 0;
    }

    uint256 underlyingBalAfter = IERC20(underlying).balanceOf(address(this));
    uint256 underlyingOut = underlyingBalAfter - underlyingBalBefore;
    if (underlyingOut == 0) {
        _refundToken(wrapper, guid, "unwrap_zero_out", refundBsc, sharesIn);
        return 0;
    }
    return underlyingOut;
}
```

Listing 2.9: src/DStockComposerRouter.sol

Impact Residual tokens from partial unwraps accumulate in the contract and cannot be recovered by users.

Suggestion Revise the logic accordingly.

Clarification from BlockSec The project team has removed the `unwrapBps` logic in Version 2, thus eliminating this issue.

2.1.6 Excessive refund due to rounding

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, the function `_wrapUnderlying()` invokes the wrapper's function `wrap()` and executes a refund logic when the minted amount (i.e., `sharesOut`) is zero. However, this could lead to an excessive refund when the wrapper's total shares exceed the underlying assets due to reverse splits. Specifically, when the `shareBalBefore` is zero, the `sharesOut` is calculated based on this formula:

wrapper's share = underlyingAmount * floor(_totalShares / availBefore)

sharesOut = shareBalAfter = Wrapper's share * floor(availBefore/_totalShares)

When the input underlying amount is small, there might be a situation where the wrapper's share is non-zero while the minted `sharesOut` is rounded down to zero. This could result in the router transferring the underlying amount to the wrapper and additionally refunding the user later (line 591).

```
585     uint256 shareBalBefore = IERC20(wrapper).balanceOf(address(this));
586     IDStockWrapperLike(wrapper).wrap(underlying, underlyingAmount, address(this));
587     uint256 shareBalAfter = IERC20(wrapper).balanceOf(address(this));
588     uint256 sharesOut = shareBalAfter - shareBalBefore;
589
590     if (sharesOut == 0) {
591         _refundToken(underlying, guid, "wrap_zero_shares", refundBsc, underlyingAmount);
592         return 0;
```

Listing 2.10: src/DStockComposerRouter.sol

Impact If the minted shares round down to zero, the router sends an extra refund to users, causing potential loss.

Suggestion Revise the code logic accordingly.

2.2 Recommendation

2.2.1 Lack of invoking function `_disableInitializers()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In contract `DStockComposerRouter`, the function `_disableInitializers()` is not invoked in the constructor. Invoking this function prevents the contract itself from being initialized, thereby avoiding unexpected behaviors.

Suggestion Invoke the function `_disableInitializers()` in the constructor.

2.2.2 Add access control to `unwrapAndPayout()`

Status Fixed in `Version 2`

Introduced by Version 1

Description In the contract WrappedNativePayoutHelper [3], the function unwrapAndPayout() is publicly accessible without any access control. If native tokens are accidentally sent to this contract, anyone can call the function unwrapAndPayout() to extract these funds by specifying themselves as the receiver. It is recommended to add access control to restrict the function unwrapAndPayout() to authorized callers only, such as the router contract.

```
function unwrapAndPayout(
    address wrappedNative,
    address receiver,
    address refundBsc,
    uint256 amount
) external returns (bool ok) {
    // 1) unwrap into native on this helper
    try IWrappedNativeLike(wrappedNative).withdraw(amount) {} catch {
        _tryTransfer(wrappedNative, refundBsc, amount);
        return false;
    }

    // 2) forward native to receiver
    (ok, ) = receiver.call{value: amount}("");
    if (ok) return true;

    // 3) receiver rejected native: re-wrap and refund as wrapped token
    try IWrappedNativeLike(wrappedNative).deposit{value: amount}() {} catch {
        // If re-wrapping fails, best-effort native refund to refundBsc.
        (bool ok2, ) = refundBsc.call{value: amount}("");
        ok2;
        return false;
    }

    _tryTransfer(wrappedNative, refundBsc, amount);
    return false;
}
```

Listing 2.11: src/WrappedNativePayoutHelper.sol

Suggestion Add access control to the function unwrapAndPayout() to restrict calls to authorized addresses.

2.2.3 Add a quoter for function wrapAndBridgeNative()

Status Fixed in Version 2

Introduced by Version 1

Description The contract DStockComposerRouter provides a quote function quoteWrapAndBridge() to estimate fees before invoking function wrapAndBridge(). However, there is no corresponding quoter for the function wrapAndBridgeNative(), which prevents users from estimating the required native fee before bridging native tokens.

[3] Note that the contract name is changed to WrappedNativePayoutHelperV2 in Version 2.

It is recommended to add a `quoteWrapAndBridgeNative()` function that estimates the required native fee without executing the operation.

```
function wrapAndBridgeNative(
    uint256 amountNative,
    uint32 dstEid,
    bytes32 to,
    bytes calldata extraOptions
) external payable nonReentrant returns (uint256 amountSentLD) {
```

Listing 2.12: src/DStockComposerRouter.sol

Suggestion Add a quoter function similar to the function `quoteWrapAndBridge()`.

2.2.4 Adopt accurate variable naming

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, the function `_unwrapShares()` is intended to partially unwrap incoming wrapper shares according to the specified ratio set by the input `unwrapBps`. However, the computed partial share amount is stored in the variable `attemptUnderlying`, which misleadingly suggests that the value is denominated in the underlying asset rather than in wrapper shares to be passed into the function `unwrap()`.

```
    // We may choose to unwrap only a fraction of shares (basis points).
    uint256 attemptUnderlying = (sharesIn * uint256(unwrapBps)) / 10_000;
    if (attemptUnderlying == 0) {
        _refundToken(wrapper, guid, "unwrap_zero_amount", refundBsc, sharesIn);
        return 0;
    }
```

Listing 2.13: src/DStockComposerRouter.sol

Suggestion Rename the variable to a shares-denominated identifier.

2.2.5 Revise the misleading annotation

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockComposerRouter`, the function `lzCompose()` receives the parameter `_oApp` from the LayerZero endpoint. The annotation states this parameter represents the "underlying token (OFT or local ERC20) that was credited to this router". However, the parameter `_oApp` represents the caller of the endpoint's function `sendCompose()`. The current annotation is inaccurate and could lead to developer confusion.

```
function lzCompose(
    address _oApp,
    bytes32 _guid,
    bytes calldata _message,
    address /*_executor*/,
```

```
        bytes calldata /*_extraData*/
    ) external payable nonReentrant {
        if (msg.sender != endpoint) revert NotEndpoint();

        // Idempotency: LayerZero may retry compose; we must not double-process the same GUID.
        if (processedGuids[_guid]) return;
        processedGuids[_guid] = true;

        // Decode the OFT compose container:
        // - `amountLD`: amount credited to this router before compose executes
        // - `inner`: our router payload (abi-encoded RouteMsg/ReverseRouteMsg)
        bytes memory inner = _message.composeMsg();
        uint256 amountLD = _message.amountLD();

        // Reverse: _oApp is a supported shareAdapter
        address wrapper = shareAdapterToWrapper[_oApp];
        if (wrapper != address(0)) {
            // Reverse path consumes shares (`amountLD`) and produces underlying.
            _lzComposeReverse(wrapper, _guid, amountLD, inner);
            return;
        }

        // Forward: _oApp is the underlying token (OFT or local ERC20) that was credited to this
            router
        address shareAdapter = underlyingToShareAdapter[_oApp];
        wrapper = underlyingToWrapper[_oApp];
        if (wrapper == address(0) || shareAdapter == address(0)) revert InvalidOApp();

        // Forward path consumes underlying (`amountLD`) and produces shares bridged via `
            shareAdapter`.
        _lzComposeForward(_oApp, wrapper, shareAdapter, _guid, amountLD, inner);
    }
```

Listing 2.14: src/DStockComposerRouter.sol

Suggestion Revise the annotation accordingly.

2.2.6 Correct length of storage gaps

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In upgradeable contracts, the array `__gap` usually has a length of `(50 - occupied slot)` to reserve space for future upgrades. The contract `DStockComposerRouter` has more than 2 slots occupied by the storage variables. However, the array `__gap` is declared with a length of 48, which does not align with the standard upgradeable storage pattern.

```
    uint256[48] private __gap;
```

Listing 2.15: src/DStockComposerRouter.sol

Suggestion Recalculate and adjust the `__gap` length to conform to the standard 50-slot layout.

2.3 Note

2.3.1 Potential centralization risks

Introduced by `Version 2`

Description In this project, several privileged roles (e.g., `ADMIN_ROLE`) can conduct sensitive operations, which introduces potential centralization risks. For example, the `ADMIN_ROLE` of contract `DStockComposerRouterV2` can modify critical parameters via functions `setRouteConfig()`, `setWrappedNative()`, and `setWrappedNativePayoutHelper()`, and extract assets via functions `rescueToken()` and `rescueNative()`. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

In addition, the `endpoint` and `chainEid` values must be configured accurately during initialization to ensure correct cross-chain message handling through LayerZero's infrastructure.

2.3.2 Correct implementation and configuration of adapters

Introduced by `Version 1`

Description In contract `DStockComposerRouter`, the bridging operations rely on the correct implementation and configuration of the OFT or OFTAdapter for both wrapper and underlying tokens. In this audit, we assume this correctness as follows:

1. The contract `DStockComposerRouter` currently uses a "lock-and-mint" mechanism for bridging wrapped tokens (i.e, `DStockWrapper`). Thus, an OFTAdapter must be deployed on the source chain, where the contract `DStockComposerRouter` is deployed. The corresponding OFT should be deployed on each supported destination chain.
2. Crucial configurations (e.g., peers for supported paths, and endpoint address for access control) must be established to ensure successful token bridging between chains.

2.3.3 Risks regarding rebasing assets

Introduced by `Version 1`

Description The protocol may integrate rebasing assets through `DStockWrapper` tokens and underlying Stock RWA tokens. When handling such assets, the protocol should account for any balance discrepancies that arise from the rebase process.

For example, in the reverse compose path, the function `sendCompose()` queues a compose operation when `DStockWrapper` tokens are transferred to the router. The operation executes via the function `lzCompose()` after several blocks. If the contract `DStockWrapper` rebases during that interval (e.g., due to holding fees), the router's effective balance decreases. However, the router still uses the original credited amount for unwrapping or processing refunds. This discrepancy leads to the router attempting to transfer more tokens than intended, resulting in transaction reverts or fund losses.

Feedback from the project The project team states that the current underlying tokens are non-rebasing, and the `DStockWrapper` rebase mechanism is disabled.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS